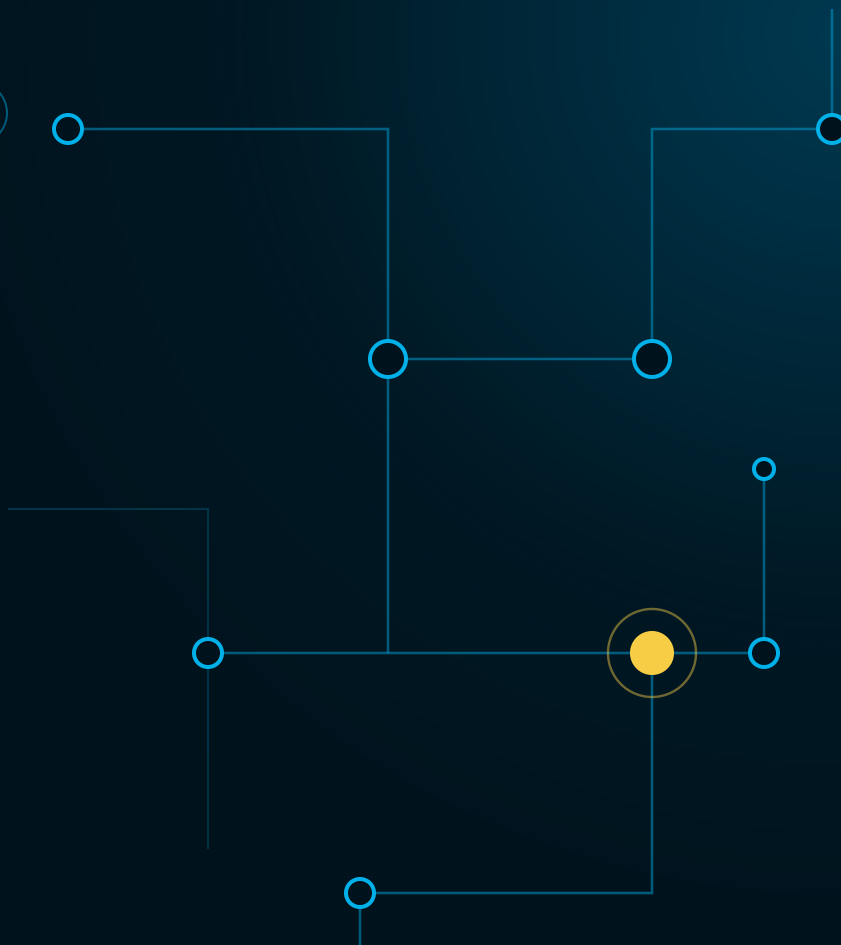




```
> git push origin main
```

ETAPA 04 / 06

Publicación:

Git, GitHub y CI/CD.

Del Vibe Coding a la Ingeniería Web
Desarrolla tu web con Claude

INSTRUCTOR

Felipe Cuevas

felipecuevas.dev

¿Por qué Git y GitHub son el idioma común de la industria?

Porque prácticamente todo el software moderno se construye con ellos, desde un portafolio personal hasta los proyectos de las empresas más grandes del mundo. Saber usarlos no es un plus: es el punto de partida.



LA HERRAMIENTA

Git

Vive en **tu computador**. Registra cada cambio que confirmas y te permite volver atrás, comparar versiones y trabajar en paralelo sin pisar el trabajo de nadie.



LA PLATAFORMA

GitHub

Vive en **internet**. Guarda tus repositorios Git en la nube, permite colaborar con otras personas y automatizar procesos.

GitHub es mucho más que un disco en la nube



La comunidad de desarrolladores más grande del mundo: ahí viven los proyectos de código abierto que usas todos los días.



Tu portafolio público de código: los reclutadores revisan tu perfil de GitHub tanto como tu CV.



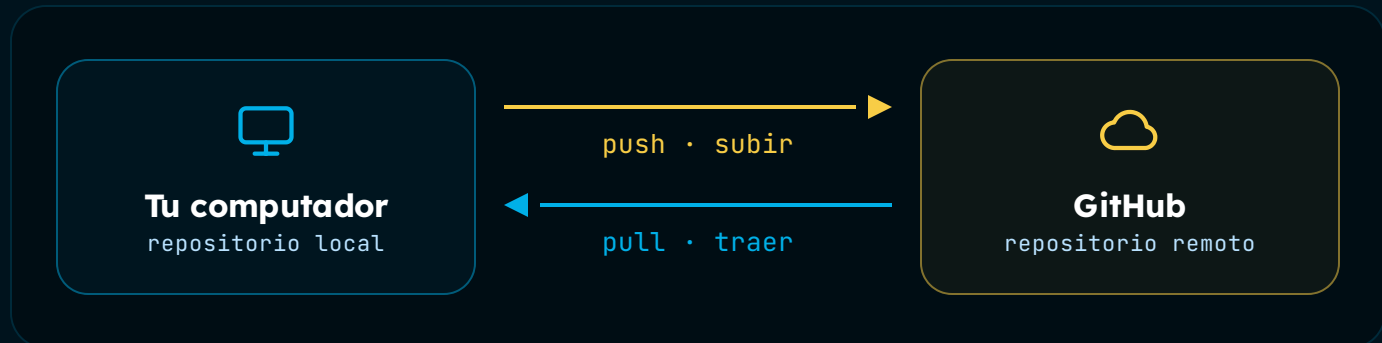
Un centro de automatización: con GitHub Pages y GitHub Actions publica y construye tu sitio por ti.



Git es tu red de seguridad. GitHub es tu vitrina ante el mundo.

¿Qué pasa realmente cuando haces **commit y push**?

Tu proyecto existe en dos lugares: tu computador (repositorio **local**) y GitHub (repositorio **remoto**). Estos comandos los mantienen sincronizados.



COMMIT · CONFIRMACIÓN

Una “foto” de tu proyecto en un momento dado, con un mensaje que explica qué cambió. Se guarda **solo en tu computador**.

PUSH · SUBIR CAMBIOS

Envía tus commits locales a GitHub. Hasta que no haces push, **nadie más los ve**.

PULL · TRAER CAMBIOS

Descarga a tu computador los cambios nuevos que existen en GitHub.

BRANCH · RAMA

Una línea paralela de trabajo. La principal se llama **main**, y es la que se publica.



Buen hábito: **commits pequeños y frecuentes**, con mensajes claros. Si algo se rompe, sabrás exactamente qué cambio revertir.



El commit guarda la foto. El push la comparte con el mundo.

¿Por qué revisar el diff es el hábito número uno?

El **diff** (diferencias) muestra exactamente qué líneas se agregaron, eliminaron o modificaron. Es la última oportunidad de detectar un error antes de que quede guardado en tu historial.

● index.html · diferencias

```
<section id="experiencia">
- <h3>Cargo de ejemplo</h3>
- <p>Empresa ficticia · 2021 - 2023</p>
+ <h3>Desarrolladora Web Junior</h3>
+ <p>Agencia Digital · 2024 - Actualidad</p>
</section>
```

+ verde: línea agregada - rojo: línea eliminada



¿Dónde lo ves en WebStorm?

En la ventana de commit (`Ctrl` + `K`), haz doble clic sobre cualquier archivo modificado: se abre una vista lado a lado con la versión anterior y la nueva.



Crucial cuando trabajas con IA

Un agente puede modificar diez archivos en segundos. Leer el diff es la forma de saber **qué cambió de verdad**, no solo lo que la IA dice que cambió. Es el hábito que separa a un Líder Técnico de alguien que acepta a ciegas.



Supervisar no es delegar a ciegas. Si no revisaste el diff, no sabes qué estás publicando.

¿Cómo pasa tu sitio de GitHub a internet?

Tu repositorio ya está en GitHub desde la Etapa 3. Ahora lo publicas con **GitHub Pages**: un servicio gratuito que convierte un repositorio en un sitio web con dirección pública.

ACTÍVALO EN 4 PASOS

- 1 En tu repositorio, entra a [Settings](#) → [Pages](#).
- 2 En [Source](#), elige [Deploy from a branch](#).
- 3 En [Branch](#), selecciona `main` y la carpeta `/ (root)`. Guarda con [Save](#).
- 4 Espera uno o dos minutos y recarga: aparecerá la dirección de tu sitio.

TU SITIO QUEDA EN

<https://tu-usuario.github.io/mi-portafolio>

Recuerda: GitHub Pages busca el archivo `index.html` en la raíz. Por eso lo renombraste en la Etapa 2.

¿Existen otras opciones?

Sí. Funcionan con la misma idea: conectas tu repositorio y publican tu sitio.

Cloudflare Pages

Red global muy rápida y dominio propio sencillo.

Vercel

Muy popular en proyectos con frameworks modernos.

Netlify

Pionero en publicar sitios estáticos desde Git.

¿Qué NO debe subir nunca a tu repositorio?

Todo lo que se puede regenerar, lo que es personal de tu computador y lo que es secreto. El archivo **.gitignore** le dice a Git qué carpetas y archivos debe ignorar. Es pequeño, pero vital.

• .gitignore

```
# Dependencias: se reinstalan con npm
node_modules/

# Configuración personal del IDE
.idea/

# Archivos del sistema operativo
.DS_Store
Thumbs.db

# Registros de errores
*.log

# Secretos y claves privadas
.env
```

NODE_MODULES/

Pesa cientos de megas y se regenera con un comando. Subirlo es un error clásico.

.IDEA/

La configuración personal de tu WebStorm. No le sirve a nadie más.

.ENV

Claves y contraseñas. Si llegan a un repositorio público, quedan expuestas para siempre en el historial.



¿Y la carpeta **_planificacion/**?

Esa sí se sube, siempre. Es la memoria del proyecto: si está versionada, Claude puede leer tus épicas, el registro-log y los RDA para auditar tu avance y retomar el trabajo donde quedó.



Un repositorio limpio dice mucho de quien lo mantiene. Los reclutadores también lo notan.

¿Qué es CI/CD y por qué te lo van a pedir?

Es la práctica de dejar que las máquinas hagan lo repetitivo: **verificar**, **construir** y **publicar** tu software automáticamente cada vez que subes cambios. Aparece en casi todas las ofertas laborales de desarrollo web.

EVENTO
git push



GITHUB ACTIONS · RUNNER
Instala → construye → publica



RESULTADO
Sitio en vivo

CI

Integración Continua

En cada cambio, una máquina **verifica** y **construye** el proyecto. Si algo falla, te enteras al tiro, no una semana después.

CD

Despliegue Continuo

Si la construcción salió bien, el sitio se **publica automáticamente**. Sin subir archivos a mano.



¿Por qué lo necesitará tu portafolio?

Hoy tu sitio no necesita construirse: Stitch lo entregó listo. Pero en la **Épica 2** vas a compilar Tailwind, y ahí aparece un paso de construcción. CI/CD hará ese trabajo por ti en cada push.



Si una tarea se repite en cada cambio, es trabajo para una máquina, no para ti.

¿Qué significan las palabras que **verás en todas partes**?

GitHub Actions tiene su propio vocabulario. Con estas tres palabras ya puedes leer cualquier flujo automatizado.



WORKFLOW · FLUJO DE TRABAJO

La secuencia de pasos automáticos que se ejecuta ante un evento, como un push a main. Vive en un archivo dentro de la carpeta `.github/workflows/`.



RUNNER · MÁQUINA DE EJECUCIÓN

Una máquina virtual temporal que GitHub levanta para tu workflow. Se crea, hace el trabajo y se elimina. Parte siempre desde cero: por eso instala todo de nuevo en cada ejecución.



YAML · FORMATO DE CONFIGURACIÓN

El formato en que se escriben los workflows. Se basa en la **indentación** (espacios al inicio de cada línea):

```
steps:
  - run: pnpm install
  - run: pnpm build
```

Ojo: un espacio de más o de menos puede romper el archivo completo.



Automatizar no es perder el control: es decidir una vez cómo se hacen las cosas, y dejar que se repitan sin errores.

¿Cómo preparas tu primer workflow?

Esta tarea corresponde a la **Épica 2 · Iteración 2**. Hazla después de la Iteración 1, cuando ya compiles Tailwind en local. Completa las partes en **dorado** con ayuda de Claude.

• `.github/workflows/publicar.yml`

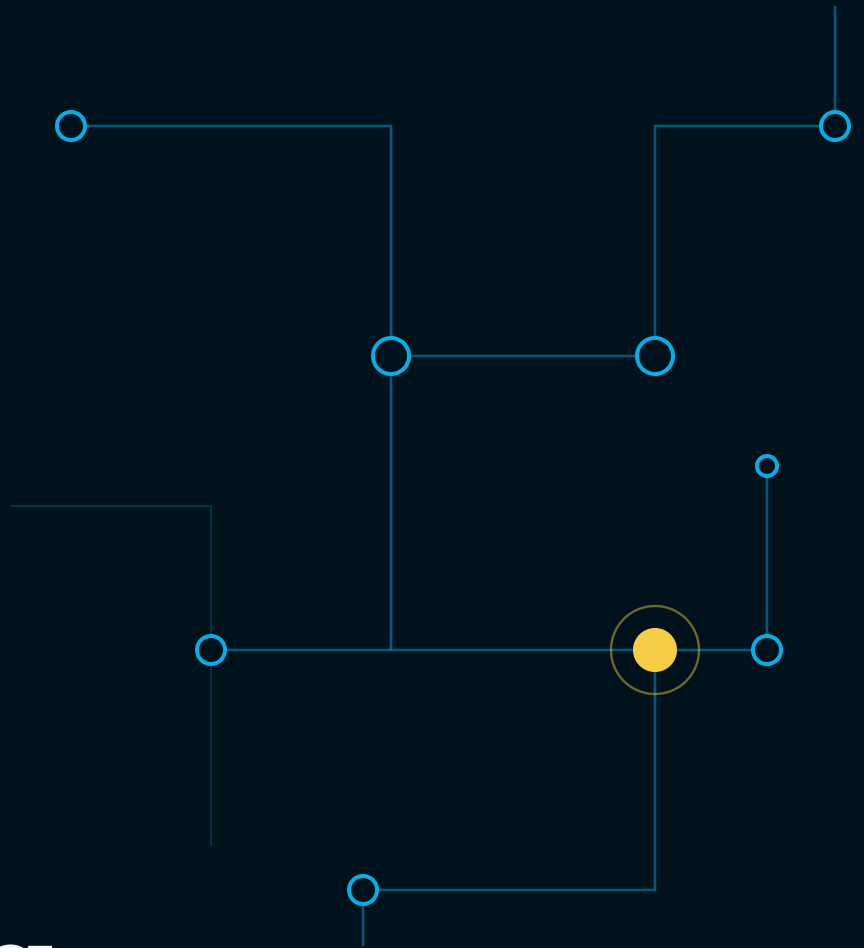
```
name: Publicar portafolio
on:
  push:
    branches: [ _____ ]          # ¿qué rama dispara la publicación?
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  construir-y-publicar:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.despliegue.outputs.page_url }
    steps:
      - uses: actions/checkout@v7
      - uses: pnpm/action-setup@v6      # pista: lee la versión de pnpm del package.json
      - uses: actions/setup-node@v7
        with:
          node-version: _____      # ¿qué versión LTS de Node.js usas?
          cache: pnpm
      - run: pnpm install
      - run: _____                  # el script de build de la Iteración 1
      - run: _____                  # ojo: el .gitignore NO aplica dentro del runner
      - uses: actions/configure-pages@v6
      - uses: actions/upload-pages-artifact@v5
        with:
          path: _____              # ¿qué carpeta publicas? (la raíz)
      - id: despliegue
        uses: actions/deploy-pages@v5
```

ANTES DEL PUSH

En **Settings** → **Pages**, cambia **Source** a **GitHub Actions**.

PÍDELE A CLAUDE

“Explicame este esqueleto línea por línea y guíame para completar las partes vacías, **sin darme la respuesta directa.**”



Tu sitio ya está en internet.

Ya entiendes cómo se publica. Ahora puedes delegarlo con criterio.

SIGUIENTE Etapa 5 · Delegar en Claude Code →